

Integration of Classy-Blocks and OpenFOAM within Blender

B C H Benjamin

Weekly Progress Report - Week 2

OpenFOAM GUI

FOSSEE, IIT Bombay

March 21, 2026

1 Introduction

2 Research Gap

3 Problem Statement

4 Geometry Extractor

5 VTK Importer

6 End-to-End Pipeline

Introduction

- ◀ **OpenFOAM:** A C++ toolbox for CFD using finite volume methods.
- ◀ **Classy-Blocks:** A Python library for procedural `blockMeshDict` generation, mimicking modern 3D CAD modeling.
- ◀ **Blender Integration:** Utilizing Blender as a frontend for geometry extraction and mesh visualization using `pyvista`.

Research Gap

- ◀ **Gap 1:** Lack of native Blender 5.x support for advanced Python meshing libraries like `classy_blocks`.
- ◀ **Gap 2:** Absence of an integrated “One-Click” solution for geometry extraction, mesh building, and quality reporting within a unified environment.

Problem Statement

- ◀ To develop a Blender Add-on that leverages `classy_blocks` and `pyvista` to automate OpenFOAM case setup, allowing users to extract geometry, generate high-quality meshes, and execute solvers without leaving the Blender interface.

Work Done: `geometry_extractor.py` Implementation

- ◀ **Core Function:** Implemented `extract_geometry(context)` — iterates scene objects, filters by `is_block`, dispatches by `block_type` (box, extrude, revolve).
- ◀ **Coordinate Preservation:** `_get_world_bounding_box(obj)` applies `matrix_world` to all 8 bound-box corners, ensuring blocks at arbitrary positions (e.g. 10,10,10) retain exact Blender world coordinates.
- ◀ **Extrude & Revolve:** Implemented face vertex extraction via `depsgraph` and world-space transformation; revolve reads angle, axis, and origin from custom properties.
- ◀ **Robustness:** All property reads use `getattr(..., default)`; unknown block types are skipped gracefully with a warning.
- ◀ **Unit Tests:** Wrote 10 unit tests in `test_geometry_extractor_unit.py` (mocked bpy) — all pass.

Work Done: `vtk_importer.py` Implementation

- ◀ **File Discovery:** `find_vtk_files(case_path)` searches VTK/ subfolder for `.vtk/.vtu` files recursively.
- ◀ **PyVista Loading:** `load_vtk_as_blender_mesh()` reads VTK via `pv.read()`, extracts the boundary surface, and creates a Blender mesh object using `from_pydata()`.
- ◀ **Face Parsing:** `_parse_pyvista_faces()` converts PyVista's flat face array to Blender tuples; handles triangles, quads, and arbitrary polygons.
- ◀ **Clean Re-import:** `_remove_existing_object()` deletes the previous mesh before reloading, preventing viewport duplication.
- ◀ **Error Handling:** Descriptive errors for unreadable files and empty surfaces; debug stats logged to System Console.
- ◀ **Unit Tests:** 15 unit tests in `test_vtk_importer_unit.py` — all pass.

Work Done: Full Pipeline Verified

- ◀ **Pipeline:** Tag cube → Generate blockMeshDict → Run blockMesh → Convert to VTK → Reload in Blender — all steps pass from the Blender UI.
- ◀ **Case Setup:** Integrated case_setup.py into the generate step — controlDict, fvSchemes, fvSolution, O/U, O/p are created automatically.
- ◀ **Result:** BlockMesh_Result object appears in viewport with correct surface geometry (602 vertices, 600 faces for a $10 \times 10 \times 10$ cube).
- ◀ **Total Automated Tests:** 29/29 passing.